

PROGRAMMATION EN LANGAGE C

Compétences à développer :

- Programmation en mode console

5

UNITE D'ENSEIGNEMENT 45 : INTRODUCTION AU LANGAGE C

Objectifs pédagogiques :

- Installer un compilateur C ;
- Ecrire la structure d'un programme C ;
- Inclure les bibliothèques `stdio.h`, `stdlib.h`, `math.h` et `conio.h` ;
- Utiliser les fonctions d'entrée/sortie classiques (`scanf`, `printf`, `get`,) ;

Contrôle de presrequis :

1. Donner les éléments essentiels d'un algorithme.
2. Savoir écrire un algorithme pour résoudre de problèmes mathématiques et physiques du niveau.

SITUATION PROBLEME :

Votre ami souhaite utiliser l'ordinateur pour exécuter ses algorithmes. Pour cela, votre petit frère lui propose la traduction de ces algorithmes en langage C avant de les exécutés. Ne connaissant rien sur ce langage, il fait donc appel à vous dans le but de l'expliquer quelques notions sur langage C.

Consignes :

1. Définir langage de programmation (**Réponse** : ensemble des mots et symboles permettant d'écrire un programme).
2. A part le langage C, quel autre langage de programmation connaissez-vous ? (**Réponse** : java, C++, pascal, python, javascript, PHP)
3. Comment appelle-t-on un algorithme déjà traduit en langage de programmation ? (**Réponse** : programme)
4. Comment appelle-t-on l'application qui permet d'exécuter un programme sur un ordinateur ? (**Réponse** : compilateur)
5. Donner la structure du programme écrit en langage C.

Réponse :

[Directives au préprocesseur]

[Déclarations de variables externes]

[Fonctions secondaires]

```
int main ()  
{
```


Déclarations de variables internes instructions

}

6. Donner le rôle des bibliothèques en langage C. Puis énumérer quelques exemples (**Réponse :**
7. Donner une fonction utilisée en langage C pour :
 - Afficher un message : **printf()**
 - Lire une variable : **scanf()**

RESUME

Définition :

Programmation : c'est la traduction d'un algorithme en un langage de programmation.

Langage de programmation : ensemble des mots et symboles permettant d'écrire un programme.

Programme : Suite d'instructions écrite dans un langage de programmation quelconque et permettant de réaliser une ou plusieurs tâches.

Installation d'un compilateur

Un **compilateur** : est une application qui transforme le code source d'un programme en un fichier binaire exécutable par la machine.

Exemple de compilateur : GNU Assembler, Turbo Pascal, Turbo C, GNU Pascal, Delphi, javac, GCJ (Gnu Compiler for Java), Jikes, Visual Basic, FreeBasic....

NB : chaque langage de programmation a son compilateur approprié. Par exemple Turbo Pascal, Delphi pour le langage **Pascal** et Javac , GCJ pour le langage **Java**.

Pour écrire des programmes, il est nécessaire d'installer différents outils. Le premier et le plus important est **l'installation d'un compilateur**.

Certains compilateurs sont déjà disponibles sur plusieurs SE, alors que d'autres sont spécifiques d'un système. Pour le plus connus sur Desktop : GCC (pour Windows, Android, Linux), Microsoft Visual Studio (MSVC).

D'autres compilateurs sont intégrés dans les environnements de développement (IDE). Il suffit de les cocher lors de l'installation de ces IDE.

Exemples des IDE: Code:: Blocks, Visual Studio, Qt Creator, Dev C++, Dev Pascal, Eclipse...

Structure générale d'un programme C

Définition :

Une expression : est une suite de composants élémentaires syntaxiquement correcte. Elle est toujours suivie d'un point-virgule (;)

En C on n'a pas une structure syntaxique englobant tout, comme la construction « **Algorithme ... Fin.** ». Un programme n'est qu'une collection de fonctions assortie d'un ensemble de variables globales. Ainsi un programme C est structuré comme suit :

```
[Directives au préprocesseur]
[Déclarations de variables externes]
[Fonctions secondaires]

int main ()
{
    Déclarations de variables internes
    instructions
}
```

La ligne : **int main()** se nomme un "**en-tête**". Elle précise que ce qui sera décrit à sa suite est en fait le "programme principal". Elle peut avoir des paramètres formels.

Le **programme (principal)** proprement dit est constitué des variables internes et des instructions et vient à la suite de cet en-tête. Il est délimité par les accolades "{" et "}".

Exemple de programme C affichant "Bonjour".

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      printf("Bonjour tout le monde !\n" ) ;
7      return 0;
8  }
```

La première ligne de notre programme : `#include <stdio.h>` est en fait un peu particulière. Il s'agit d'une "directive" qui est prise en compte avant la traduction (compilation) du programme.

Ces directives, contrairement au reste du programme, doivent être écrites à raison d'une par ligne et elles doivent obligatoirement commencer en début de ligne. Leur emplacement au sein du programme n'est soumis à aucune contrainte (mais une directive ne s'applique qu'à la partie du programme qui lui succède). D'une manière générale, il est préférable de les placer au début.

La directive demande en fait d'introduire (avant compilation) des instructions (en langage C) situées dans le fichier `stdio.h`. Notez qu'un même fichier en-tête contient des déclarations relatives à plusieurs fonctions. En général, il est indispensable d'incorporer `stdio.h`.

Quelques bibliothèques (ou directives de préprocesseur) utilisés en langage C sont :

Bibliothèques	Rôle
<stdio.h>	Fournit la capacités centrales d'entrées/sorties

<math.h>	Pour calculer des fonctions mathématiques
<stdlib.h>	Pour exécuter les opérations de conversion, l'allocation des mémoires, le contrôle de processus, le tri, la recherche, ...
<string.h>	Pour manipuler les chaines de caractères

NB : cette liste des bibliothèques standard C n'est pas exhaustive.

Notion des variables et operateurs

✓ **Variable**

Une variable est un objet manipulé dans l'exécution d'un programme. Elle est caractérisée par son **identificateur** (nom), son **type** et parfois sa **valeur** (le cas de constante).

Quelques types prédéfinis en langages C sont :

- **int** (*entier naturel*)
- **Char** (*caractères*)
- **float , double** (*nombre à virgule*)

Une variable doit être déclarée avant son utilisation. En C, toute instruction composé d'un spécificateur de type et d'une liste d'identificateurs séparés par ne virgule est une **déclaration**. Ainsi en C, une variable est déclarée de la manière suivante : *type identificateur ;*

Exemples : *int a ;* et *char a ;* sont de déclarations

✓ **Opérateur**

Le tableau ci-dessous résume quelques operateurs utilisés en langage C

Opérateurs	Symboles	Opérateurs	Symboles
Affectation	=	égal	==
Reste de la division(modulo)	%	diffèrent	!=
Multiplication	*	ET logique	&&
Inférieur ou égal	<=	OU logique	

Les entrées/sorties en C

Durant l'exécution d'un programme, le processeur, qui est le cerveau de l'ordinateur, a besoin de communiquer avec le reste du matériel. Il doit en effet recevoir des informations pour réaliser des actions et il doit aussi en transmettre. Ces échanges d'informations sont les entrées et les sorties (ou input / output en anglais).

• **Les sorties**

Le tableau ci-dessous décrit les trois fonctions d'affichage de données

Fonctions	Rôle
Printf()	Pour écrire une chaîne de caractères formatée
puts ()	Pour écrire une chaîne de caractères toute simple
Putchar()	Pour écrire un caractère

La syntaxe de l'utilisation de ces fonctions est :

fonction ("texte_ a_afficher... ") ;

Exemples :

```
printf("Bonjour le monde");  
  
puts("Hello world !") ;  
  
put("c");
```

La fonction **Printf** permet non seulement d'afficher des chaînes de caractères simples, mais également la valeur d'une variable passée en paramètre. Pour ce faire, il suffit d'utiliser un indicateur de conversion : il s'agit du caractère spécial % suivi d'une lettre qui varie en fonction du type de la variable.

Le tableau ci-dessous donne quelques indicateurs de conversion :

Type	Indicateurs de conversions
char	%c
int	%d
long	%ld
short	%hd
float	%f
double	%f
long double	%Lf
unsigned int	%u
unsigned short	%hu
unsigned long	%lu

Après avoir inscrit un indicateur de conversion dans la chaîne de caractère (dans les guillemets ""), il faut indiquer de quelle variable il faut afficher la valeur. Il suffit de rajouter une virgule après les ces derniers, suivis du nom de la variable, comme ceci :

printf ("% [lettre]", variable_ a_afficher);

Exemples :

```
1  #include <stdio.h>  
2  
3  
4  int main(void)  
5  {  
6      int variable = 20;  
7  
8      printf("%d\n", variable);  
9      return 0;  
10 }
```

Remarque : Plutôt qu'appeler plusieurs fois la fonction printf pour écrire du texte, on peut ne l'appeler qu'une fois et écrire plusieurs lignes. Pour cela, on utilise le signe \ à chaque fin de ligne.

- **Les entrées**

Pour récupérer les valeurs saisies par l'utilisateur, nous utilisons les fonctions suivantes : **scanf ()** et **get ()**

La syntaxe de l'utilisation de la fonction **Scanf()** est :

scanf("%[lettre]", &variable qui_contiendra_notre_valeur);

NB : La fonction **scanf** a besoin de connaître l'emplacement en mémoire de nos variables afin de les modifier. Afin d'effectuer cette opération, on utilise le symbole **&**. Donc, si vous oubliez le **&**, le programme plantera quand vous le lancerez, car vous tenterez d'accéder à une adresse mémoire inexistante !

Exemple :

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int age;
7
8      printf("Quel âge avez-vous ? ");
9      scanf("%d", &age);
10     printf("Vous avez %d an(s)\n", age);
11     return 0;
12 }
```

SITUATION D'INTEGRATION

Vous souhaitez écrire un programme C qui demande à l'utilisateur de saisir un nombre puis affiche ce nombre.

1. Donner la structure d'un programme C
2. Quelles sont les bibliothèques standard que vous avez besoin dans votre programme C
3. Ecrire l'algorithme qui permet de réaliser cette tâche.
4. Traduire votre algorithme en langage C

REINVESTISSEMENT

Dans le cadre d'un TP sur la programmation, on vous demande d'écrire un programme C qui calcule la somme de deux nombres.

1. Démarrez votre éditeur de texte puis écrire le programme demandé.
2. Enregistrez votre programme sous le nom « **addition.c** »



UNITE D'ENSEIGNEMENT 46 : LES STRUCTURES DE CONTROLE

Objectifs pédagogiques :

- Ecrire la syntaxe des structures for, while et do...while en C;
- Utiliser l'opérateur conditionnelle (?) et virgule ;

Contrôle des prérequis :

1. Donner la structure d'un programme C
2. Donner les fonctions C utilisés pour afficher un texte à l'écran et pour lire une donnée fournie par l'utilisateur.

SITUATION PROBLEME :

Votre amie souhaite écrire un programme C qui calcule et affiche la table de multiplication d'un nombre positif saisi par l'utilisateur. Elle souhaite donc que ce programme affiche à chaque fois un message à l'utilisateur s'il saisi un nombre négatif. Rencontrant des problèmes sur l'écriture de ce programme, elle donc appel à vous dans le but de l'aider.

Consignes :

1. Qu'entend-on par structure de contrôle ? (**Réponse** : est une instruction particulière d'un langage de programmation impératif pouvant dévier le flot de contrôle du programme la contenant lorsqu'elle est exécutée).
2. Enumérer les différentes structures de contrôle utilisées en programmation. (**Réponse** : structure alternative, structure itérative)
3. Quelle structure utilise-t-on dans le cas où la condition se traite en deux cas possibles ? (**Réponse** : structure alternative).
4. Quelle boucle utilise-t-on dans chacun des cas suivants :
 - La répétition connue d'un bloc instructions d'un programme ? (**Réponse** : le boucle pour...)
 - La répétition non connue d'un bloc instructions d'un programme ? (**Réponse** : boucle tant que...).

RESUME

En programmation informatique, une **structure de contrôle** est une instruction particulière d'un langage de programmation impératif pouvant dévier le flot de contrôle du programme la contenant lorsqu'elle est exécutée. Le langage C propose plusieurs structures de contrôle différentes, qui nous permettent de nous adapter à tous les cas possibles. Il permet d'utiliser les conditions (structures alternatives), des boucles (structures itératives).

• Structure alternative (if...else...)

Il s'agit de tester une condition et d'effectuer une série d'instructions si la condition est vraie et si elle est fausse, effectuer une autre série d'instructions.

Il existe deux formes de structures conditionnelles :

- **La forme réduite**

```
if (condition) {  
  
    bloc instructions 1  
  
}
```

Exemple : le programme C qui prend en entrée un nombre puis affiche le message « positif » si c'est ce nombre est supérieur à 0 est :

```
#include <Stdio.h>  
int main (void) {  
    int n ;  
    printf ("saisir un nombre");  
    scanf ("%d",&n);  
    if (n>0) {  
        printf("Positif");  
    }  
    Return 0;  
}
```

- **La**

forme complète

```
if (condition) {  
  
    bloc instructions 1  
  
} else {  
  
    bloc instructions 2  
  
}
```

Exemple : le programme C qui permet d'étudier la parité d'un nombre saisi par l'utilisateur est :

```
#include <Stdio.h>  
int main (void) {  
    int n ;  
    printf ("saisir un nombre");  
    scanf ("%d",&n);  
    if (n%2==0) {  
        printf("%d",n,"est paire");  
    } else {  
        printf("%d",n,"est impaire");  
    }  
    Return 0;  
}
```

- **L'opérateur conditionnel ?**

L'opérateur conditionnel ? est un opérateur ternaire. Sa syntaxe est la suivante : **condition ? expression 1 : expression 2 ;**

Cette expression est égale à **expression 1** si **condition** est satisfaite, et à **expression 2** sinon.

Exemple : le programme C qui calcule la valeur absolue d'un nombre est :

```
#include <Stdio.h>
int main (void) {
    int x, abs;
    abs=((x>=0)? x : -x) ;
    printf(" abs= %d",abs);
}
Return 0;
}
```

• L'opérateur virgule

Une expression peut être constitué d'une suite d'expressions séparées par des virgules :

expression 1, expression 2 , , expression n ;

Cette expression est alors évaluée de gauche à droite. Sa valeur sera la valeur de l'expression de droite. Par exemple, le programme

```
#include <Stdio.h>
int main (void) {
    int a, b ;
    b=((a=3),(a+2)) ;
    printf(" b= %d",b);
}
Return 0;
}
```

retournera 5 à la sortie.

NB : la virgule séparant les arguments d'une fonction ou les déclarations des variables n'est pas l'opérateur virgule.

• Structures itératives

Toutes les structures itératives répètent l'exécution de traitement(s). Deux cas sont cependant à envisager, selon que :

- Le nombre de répétitions est connu à l'avance : c'est le cas des boucles itératives.
- Le nombre de répétitions n'est pas connu ou est variable : c'est le cas des boucles conditionnelles.

○ Boucle for

Cette structure est une boucle itérative ; elle consiste à répéter un certain traitement un nombre de fois fixé à l'avance. Cette structure est donnée par :

```
for (initialisation ; condition ;incrémentation ) {  
  
    bloc instructions;  
  
}
```

Exemple : Affichage de nombres plus petit ou égal à 5.

```
for (i=0 ;i<=5 ; i++ ) {  
  
    printf(" i=%d",i);  
  
}
```

- **Boucle while**

Parfois, pour réaliser une tâche, on doit effectuer plusieurs fois les mêmes instructions, sans que le nombre de fois soit déterminé à l'avance. On utilise alors une boucle conditionnelle. Dans cette structure, le même traitement est effectué tant qu'une condition reste valide ; la boucle s'arrête quand celle-ci n'est plus remplie. Cette structure répétitive est ainsi formulée :

```
while (condition vraie) {  
  
    bloc instructions;  
  
}
```

Exemple : affichage des nombres plus petit que 5

```
i=0 ;  
while (i < 5) {  
    printf("i=%d",i);  
    i++;  
}
```

- **Boucle do...while**

Une variante de structure répétitive avec boucle conditionnelle consiste à répéter un traitement jusqu'à ce qu'une certaine condition soit vérifiée. Dans ce cas, la boucle est exécutée au moins une fois, après quoi on teste la condition. On la traduit par l'instruction :

```
do {  
  
    bloc instructions;  
  
} while (condition);
```


Exemple : affichage des nombres plus petit que 5

```
i=0;
do {
printf("i=%d",i) ;
i++;
} While (i<5) ;
```

SITUATION D'INTEGRATION

Soit le programme suivant :

```
#include <stdio.h>
int main(void)
{
    int i,n,som;
    som = 0;
    for(i=0;i<4;i++)
    {
        printf(''donnez un entier '');
        scanf('' %d '',&n);
        som+=n;
    }
    printf(''somme : %d\n'',som);
    return 0 ;
}
```

Ecrire un programme C réalisant exactement la même chose, en utilisant :

- a) Une instruction while;
- b) Une instruction do ... while

REINVESTISSEMENT :

Soit le programme C ci-dessous :

```
#include <Stdio.h>
int main (void) {
int a,b,m;
m =((a>b)? a : b) ;
printf("%d",m);
}
Return 0;
}
```

a. Donner la valeur de la variable **m** dans chacun des cas suivants :

a	b	m
1	0	
23	24	
15	5	

b. Traduire ce programme en utilisant la structure alternative (if...else).

UNITE D'ENSEIGNEMENT 47 : LES TABLEAUX EN LANGAGE C

Objectif pédagogique :

- Déclarer un tableau en C ;

SITUATION PROBLEME :

Votre camarade Ali souhaite écrire un programme C qui calculera la moyenne arithmétique de 50 nombres. Votre petit frère lui conseille d'utiliser le tableau dans son programme pour stocker les 50 nombres. Ne connaissant pas comment utiliser le tableau en C, il fait appel à vous dans le but de l'aider.

Consignes :

1. Définir tableau (**Réponse** : un tableau est un ensemble fini d'éléments de même type stocké en mémoire)
2. Donner l'intérêt de l'utilisation d'un tableau. (**Réponse** : Le tableau permet la sauvegarde des données de manière structurée).
3. Donner les caractéristiques d'un tableau (**Réponse** : l'identificateur du tableau, le type des éléments du tableau et sa taille c'est-à-dire le nombre d'éléments)
4. Donner la syntaxe de déclaration d'un tableau en langage C (**Réponse** : type nom-tableau [taille-tableau]).
5. Expliquer comment, on remplit un tableau avec ses éléments en C (**Réponse** : affecter à chaque case du tableau une valeur)
6. Expliquer comment on accède aux différents éléments d'un tableau en C (**Réponse** : on accède à l'élément du tableau de la façon suivante : nom_tableau [indice_élément])

RESUME

Déclaration d'un tableau

Un tableau est un ensemble fini d'éléments de même type, stockés en mémoire à des adresses contiguës.

La déclaration d'un tableau à une dimension se fait de la façon suivante :

type nom-du-tableau[nombre-éléments] ;

où ***nombre-éléments*** est une expression constante entière positive. Par exemple :

int tab[10] ; indique que ***tab*** est un tableau de 10 éléments de type ***int***.

Pour plus de clarté, il est nécessaire de donner un nom à la constante ***nombre-éléments*** par une directive au préprocesseur, par exemple

define nombre-éléments 10

De façon similaire, on peut déclarer un tableau à plusieurs dimensions. Par exemple pour un tableau à deux dimensions :

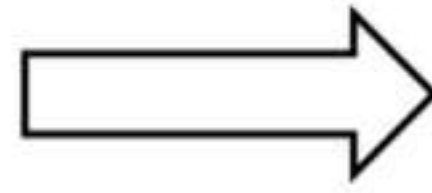
type nom-du-tableau[nombre-lignes] [nombre-colonnes] ;

Exemple : ***int tab[10][8]*** est un tableau à deux dimensions de 10 lignes et de 8 colonnes.

Ajout des éléments d'un tableau

Pour remplir les différents éléments du tableau, on pourra affecter à chaque case de ce tableau une valeur.

Exemple : `int tab[2] ;`
 `tab[0]=10 ;`
 `tab[1]= 9 ;`



10	9
----	---

Evidemment, à ce rythme-là, l'affectation est longue, surtout si votre tableau est de grande taille. C'est pourquoi on utilise la boucle itérative **for**.

On peut initialiser le tableau lors de sa création par une liste de constantes de la façon suivante :

`type nom-du-tableau[N]={constante-1,...,constante-N} ;`

Par exemple, on peut écrire :

`#define N 4`

`int tab[N]={1,2,3,4} ;`

1	2	3	4
---	---	---	---

Accès aux éléments d'un tableau

On accède à un élément du tableau en lui appliquant l'opérateur []. Les éléments d'un tableau sont toujours numérotés de **0** à ***nombre-éléments-1***

Par exemple l'instruction **`printf("%d",tab[2]) ;`** affiche le troisième élément du tableau (contenu du tableau à l'indice i=2).

On pourra afficher tous les éléments du tableau en utilisant la boucle **for**. Le programme ci-dessous réalise la tâche.

```
#include <Stdio.h>
#define N 10
int main () {
    int tab[N];
    int i;
    .....
    for (i=0;i<N;i++) {
        printf("tab[%d]=%d\n",I,tab[i]);
    }
    return 0;
}
```

SITUATION D'INTEGRATION

Ecrire un programme C qui calcule la moyenne de 5 nombres saisi par l'utilisateur.

REINVESTISSEMENT

Ecrire un programme C permettant de créer le tableau ci-dessous et de rechercher dans ce tableau une valeur fournie par l'utilisateur. Si la valeur est trouvée, alors le message suivant est affiché : « Succès ! ». Dans le cas contraire affiché « Echec ! ».

Tableau :

1	12	10	5	3
---	----	----	---	---